

12주차: 정형데이터에서의 딥러닝 활용 TabNet & TabPFN

TabNet: Attentive Interpretable Tabular Learning

arxiv.org

<https://arxiv.org/pdf/1908.07442>

Introduction

배경 및 문제의식

왜 표 형식 데이터에 딥러닝이 어려울까?

- 현실에서 가장 흔한 데이터 유형임에도 불구하고, 딥러닝보다 의사결정트리 기반 앙상블 모델이 여전히 지배적
- 기존 DNN(CNN, MLP 등)은 표 형식 데이터에 적합한 귀납적 편향이 없어 과모수화 문제 발생

과모수화 문제란?

모델이 필요이상으로 복잡해서 오히려 성능이 나빠지는 현상

그럼에도 딥러닝을 쓰려운 이유는 뭘까?

- 대용량 데이터에서 성능 향상 기대
- 이미지 + 표 데이터 같은 다중 데이터 타입 통합 처리 가능
- 피처 엔지니어링 부담 감소
- 스트리밍 데이터 학습, 준지도학습, 생성 모델 등 다양한 응용 가능

해결방안

TabNet

1. 원시 데이터를 전처리 없이 입력받아 end-to-end 학습 가능
2. 순차적 어텐션으로 각 단계마다 중요한 feature를 선택 → 효율적 학습 + 해석 가능성 확보

3. 인스턴스별 feature 선택 (압력마다 다른 feature 집합 선택 가능) + 로컬/글로벌 해석 가능성 제공
4. 비지도 사전학습 도입 - 마스킹된 feature를 예측하는 방식으로 레이블 없는 데이터 활용 시 성능 대폭 향상

Related Works

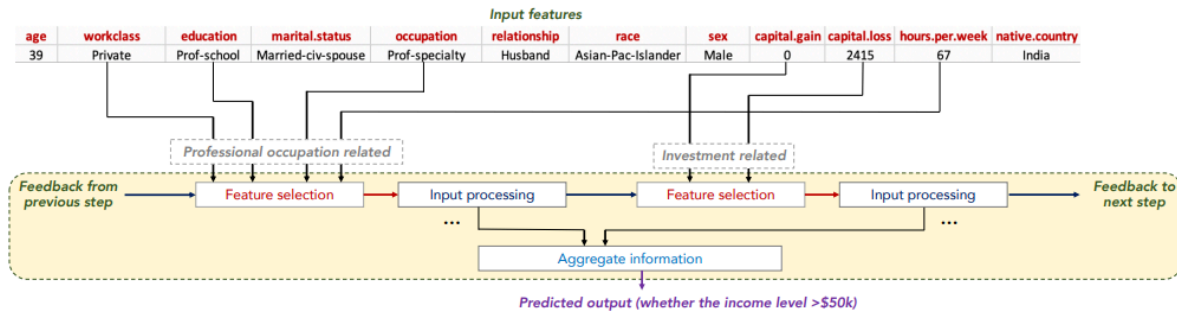


Figure 1: TabNet's sparse feature selection exemplified for Adult Census Income prediction (Dua and Graff 2017). Sparse feature selection enables interpretability and better learning as the capacity is used for the most salient features. TabNet employs multiple decision blocks that focus on processing a subset of input features for reasoning. Two decision blocks shown as examples process features that are related to professional occupation and investments, respectively, in order to predict the income level.

Feature Selection

전통적 방법 (Global)

- 전진 선택법(Forward Selection), Lasso 정규화 등
- 전체 훈련 데이터 기준으로 피쳐 중요도를 일괄 부여
- 모든 입력에 동일한 피쳐 집합 적용 → 획일적

인스턴스별 방법 (Instance-wise)

- 입력마다 개별적으로 피쳐를 선택하는 방식
- 기존 연구들은 별도의 설명 모델(explainer) 또는 actor-critic 프레임워크를 추가로 사용 → 구조가 복잡해짐

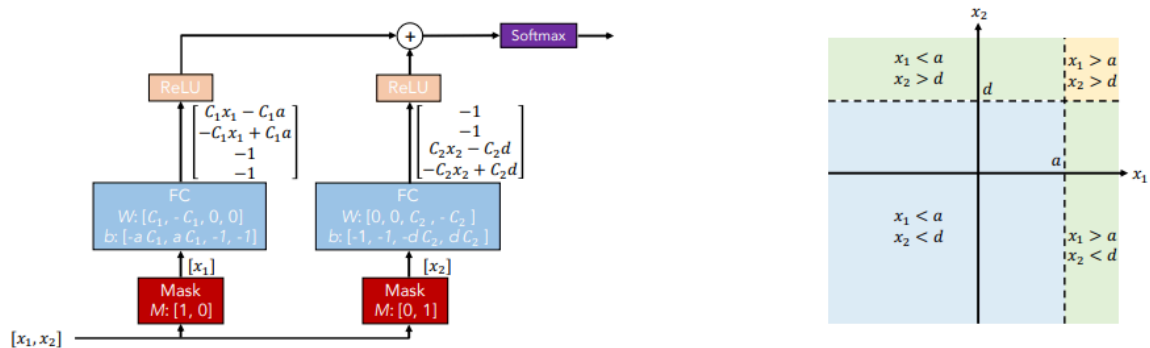
TabNet의 차별점

- 소프트 피쳐 선택(Soft Feature Selection) + 제어 가능한 희소성(Controllable Sparsity)
- 피쳐 선택과 예측을 단일 모델에서 동시에 수행 (end-to-end)
- 더 간결한 표현으로 더 높은 성능 달성

Tree-based learning

- 의사결정 트리(DT) - 통계적 정보 이득 기반으로 글로벌 feature 효율적 선택
- 랜덤 포레스트 - 데이터/피쳐 무작위 샘플링으로 여러 트리 앙상블 → 분산 감소
- XGBoost/LightGBM - 최근 데이터 경쟁 압도적 지배

⇒ TabNet은 딥러닝의 표현 용량(representation capacity) 을 높이면서도 트리의 피쳐 선택 특성을 유지해 트리 모델을 능가할 수 있음을 실험으로 증명



좌측 그림은 conventional DNN blocks 이다. 좌측 mask block을 보면 첫 번째 계수만 1이어서 첫 번째 변수들에 대해서만 학습하고, 우측 mask block에서는 두번째 변수들에 대해서만 학습한다. 그 결과 선택된 변수들로 결정 경계를 만들어가고, 이를 통해 딥러닝 모델이 트리 모델처럼 학습이 가능하다.

Integration of DNNs into DTs

연구	방식	한계
Humbird et al.	DT를 DNN 블록으로 표현	표현 중복, 비효율적 학습
Soft/Neural DT	미분 가능한 결정 함수 사용	자동 피쳐 선택 기능 상실 → 성능 저하
Yang et al.	소프트 비닝 함수로 DT 시뮬레이션	모든 가능한 결정을 열거 → 비효율적
Ke et al.	피쳐 조합을 명시적으로 활용	Gradient Boosted DT에서 지식 전달에 의존
Tanno et al.	원시 블록에서 적응적으로 성장	구조가 복잡

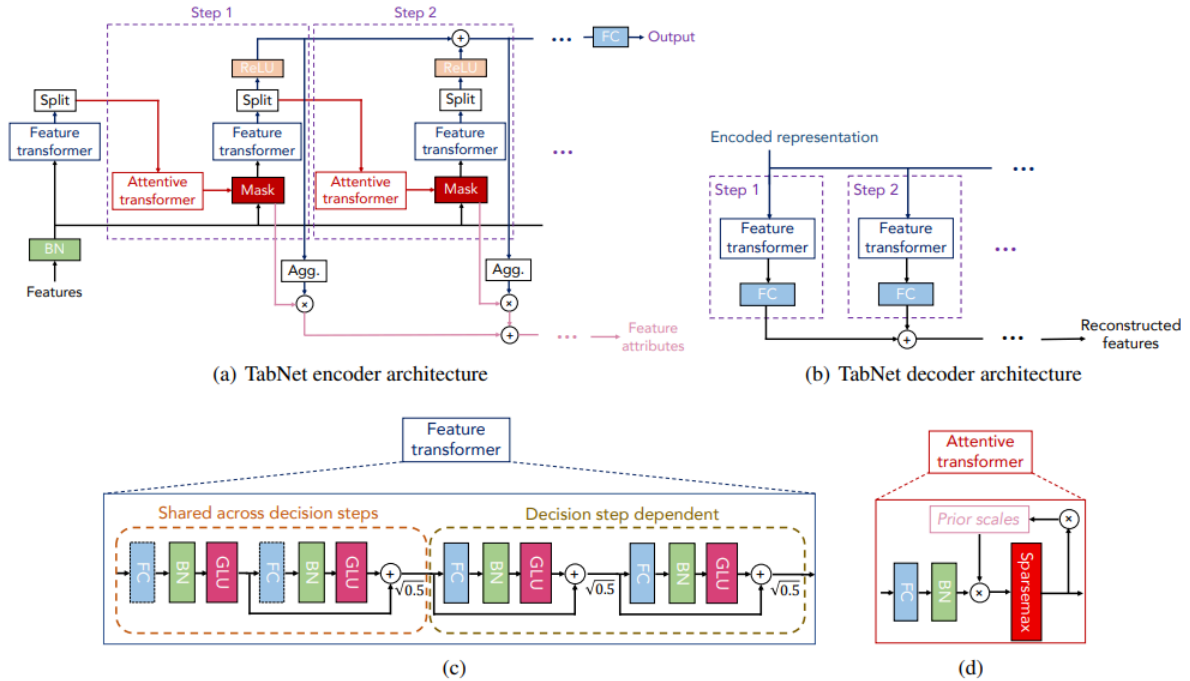
TabNet의 차별점 → 위 방법들과 달리 **순차적 어텐션(Sequential Attention)** 을 통해 희소성이 제어되는 소프트 피쳐 선택을 자연스럽게 내재화

Self-supervised learning

- 비지도 표현 학습은 특히 데이터가 적을 때 지도학습 성능을 크게 향상
- 텍스트(BERT)와 이미지 분야에서 마스킹된 입력 예측 방식이 큰 성과를 거둠

⇒ TabNet은 이 아이디어를 표 형식 데이터에 최초로 적용

Structure



전체적인 구조

Attentive Transformer

Feature Selection을 수행하는 구조로 학습 가능한 Mask를 출력한다.

이전 step에서 가공되어 input된 $a[i-1]$ 이 FC(Fully Connected) layer, BN(Batch Normalization) layer를 거치며 $hi(a[i-1])$ 이 되고, $P[i-1]$ 을 곱해준 다음 Sparsemax 적용 해 준다. 이 때, $P[i-1]$ 은 이전 step의 Prior Scales로 이전 decision step에서 선택되었던 feature의 재선택 비율을 조정해준다.

$$P[i] = \prod_{j=1}^j (\gamma - M[j])$$

$M[j]$ 는 이전 step에서 생성된 mask고, γ 는 조정 가능한 하이퍼 파라미터로 1 이상인 수이다. 이 때 γ 값을 1로 설정하면 이전 단계에서 중요했던 컬럼의 마스크 값은 1이었기에 빼주면 0이 되고 이를 다음 step의 $hi(a[i-1])$ 에 곱해주게 되므로, 이전에 선택되었던 변수는 다시 선택되지 않을 확률이 높아진다.

Feature Transformer

Feature processing을 수행하는 구조로 마스킹 과정을 거친 masked feature들을 embedding 해 준다.

첫 step의 input은 Batch Normalization을 거친 feature이고, 그 이후 step의 input은 masking 과정을 거친 feature들이다.

Feature Transformer는 두 종류의 블록으로 구성되는데, 좌측의 Shared across decision steps와 우측의 Decision step dependent이다. 좌측은 전체 decision step에서 파라미터를 공유하고, 우측은 해당 step에서의 파라미터만 이용한다.

각 block은 공통적으로 Fully Connected Layer - Batch Normalization - Gated Linear Unit 연결 구조를 지닌다. 이 때 input features를 제외한 BN은 Ghost Batch Normalization을 진행하여 학습 속도를 향상시켰고, 블록마다 루트 0.5 곱해주어 분산을 작게 해준다. 그 결과 D차원으로 임베딩 된 features(f)를 출력한다.

Split & ReLU

이렇게 결과로 나온 f 를 $d[i]$, $a[i]$ 로 Split 하여 $d[i]$ 에는 ReLU를 적용하는데 그 결과는 $step[i]$ 에서의 feature importance를 의미한다. $a[i]$ 는 다음 step을 위한 input이 된다. 최종적으로 ReLU를 적용한 $d[i]$ 들을 합해주어 최종적인 feature attributes를 생성한다.

Decoder

Self-Supervised Learning을 위해 필요한 구조로 Encoder의 각 step마다 feature transformer의 output인 $d[i]$ 를 input으로 받아서 Feature Transformer, FC layer를 거쳐서 모든 step의 결과를 최종적으로 합한 features를 출력한다. 이러한 encoder-decoder 구조를 통해 Unsupervised pre-training과정을 진행하고, 그 결과를 가지고 Supervised fine-tuning을 진행하여 성능을 향상시켰다.

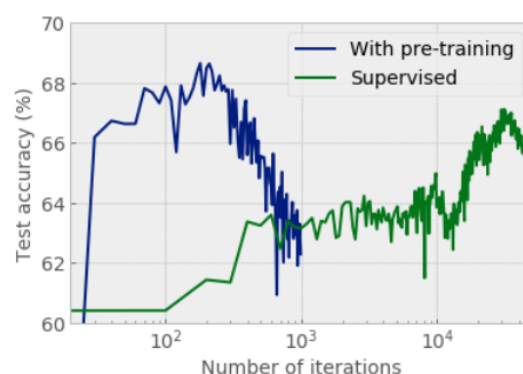


Figure 7: Training curves on Higgs dataset with 10k samples.

pre-training을 진행하지 않은 경우보다 훨씬 빠르게 수렴하는 결과를 보인다.

Interpretability

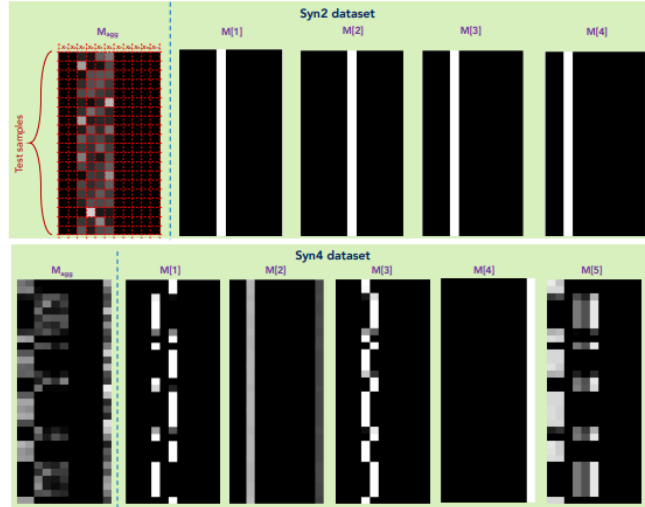


Figure 5: Feature importance masks $M[i]$ (that indicate feature selection at i^{th} step) and the aggregate feature importance mask M_{agg} showing the global instance-wise feature selection, on Syn2 and Syn4 (Chen et al. 2018). Brighter colors show a higher value. E.g. for Syn2, only X_3-X_6 are used.

위의 Syn2 데이터 셋은 샘플(행) 별로 중요 feature의 차이가 없는 데이터 셋이고, Syn4 데이터 셋은 샘플 별로 중요 feature가 다른 데이터 셋이다. Syn4 결과에서는 행 마다 흰 부분인 열(중요한 feature)이 다른 것을 확인할 수 있다. 따라서 각 step 별로 중요 feature들을 확인할 수 있고, 이를 aggregate하여 전체적인 feature 중요도 또한 확인할 수 있다.

Experiments

Table 1: Mean and std. of test area under the receiving operating characteristic curve (AUC) on 6 synthetic datasets from (Chen et al. 2018), for TabNet vs. other feature selection-based DNN models: No sel.: using all features without any feature selection, Global: using only globally-salient features, Tree Ensembles (Geurts, Ernst, and Wehenkel 2006), Lasso-regularized model, L2X (Chen et al. 2018) and INVASE (Yoon, Jordon, and van der Schaar 2019). Bold numbers denote the best for each dataset.

Model	Test AUC					
	Syn1	Syn2	Syn3	Syn4	Syn5	Syn6
No selection	.578 ± .004	.789 ± .003	.854 ± .004	.558 ± .021	.662 ± .013	.692 ± .015
Tree	.574 ± .101	.872 ± .003	.899 ± .001	.684 ± .017	.741 ± .004	.771 ± .031
Lasso-regularized	.498 ± .006	.555 ± .061	.886 ± .003	.512 ± .031	.691 ± .024	.727 ± .025
L2X	.498 ± .005	.823 ± .029	.862 ± .009	.678 ± .024	.709 ± .008	.827 ± .017
INVASE	.690 ± .006	.877 ± .003	.902 ± .003	.787 ± .004	.784 ± .005	.877 ± .003
Global	.686 ± .005	.873 ± .003	.900 ± .003	.774 ± .006	.784 ± .005	.858 ± .004
TabNet	.682 ± .005	.892 ± .004	.897 ± .003	.776 ± .017	.789 ± .009	.878 ± .004

Syn 1~6 은 저자들이 임의로 생성한 데이터 셋

⇒ 위 테이블에서 Syn 1~3은 샘플 별로 중요 feature 차이가 없는 데이터 셋이고, Syn 4~6은 샘플 별로 중요 feature 차이가 있는 데이터 셋이다. 위의 정확도 결과는 중요 feature 차이가 없는 데이터 셋에서는 다른 모델들과 비교했을 때 큰 성능 차이가 없지만, 그렇지 않은 데이터 셋에서 좋은 성능을 띠는 것을 보여주고 있다.

Accurate predictions on small data with a tabular foundation model

<https://www.nature.com/articles/s41586-024-08328-6>

Abstract

배경

- 표 형식 데이터는 생의학, 입자물리학, 경제학, 기후과학 등 **거의 모든 과학 분야**에서 사용되는 가장 보편적인 데이터 형태
- 핵심 예측 과제는 **누락된 레이블 값을 나머지 컬럼으로 채우는 것** (결측값 예측, 분류/회귀)
- 딥러닝이 이미지·텍스트에서 혁명을 일으켰음에도, **표 형식 데이터에서는 20년간 Gradient Boosted Decision Tree가 지배**

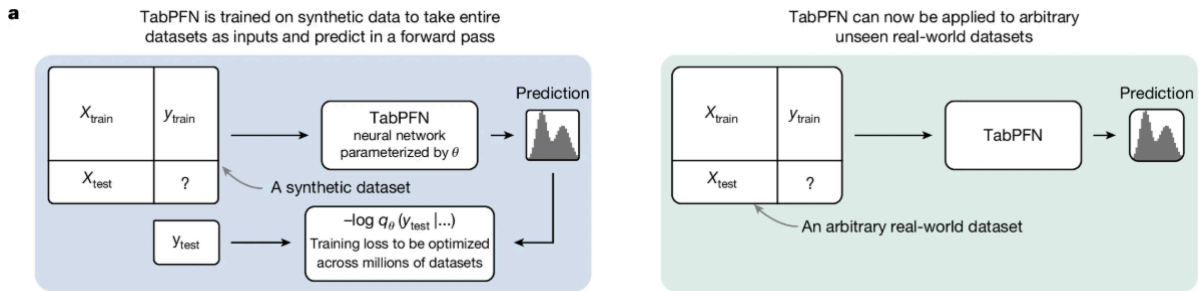
해결방안

TabPEN

- 모델 유형 : 트랜스포머 기반 생성형 파운데이션 모델
- 학습 방식 : 수백만 개의 합성 데이터셋을 통해 학습된 메타 학습 알고리즘
- 적용 범위 : 최대 10000개 샘플 데이터셋
- 핵심 성능 : 4시간 동안 추닝한 최강 베이스라인 앙상블을 단 2.8초만에 능가
 - 기존 모든 방법론 대비 압도적 성능 + 획기적인 속도
 - 별도의 학습/튜닝 시간 거의 불필요

Principled in-context learning

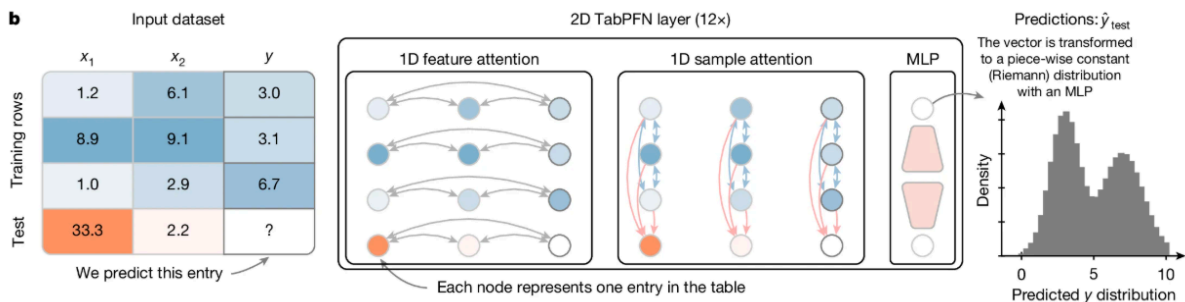
- **사전 훈련**: 수백만 개의 다양한 합성 데이터셋을 통해 '어떤 데이터가 주어져도 예측을 잘하는 일반적인 학습 알고리즘' 자체를 트랜스포머에 내재화시킨다.
(이 과정은 단 한 번만 수행됩니다)
- **추론**: 실제 데이터셋을 만나면, 훈련 데이터와 테스트 데이터를 한 번에 입력받아, 단 한 번의 순전파(forward pass)로 데이터의 패턴을 파악하고 예측까지 완료한다.



명령어로 알고리즘을 설계하는 대신, 원하는 행동을 보여주는 수많은 입출력 예시(합성 데이터)를 만들어 모델이 그 패턴을 만족하는 알고리즘을 스스로 배우게 한다.

An architecture designed for tables

구조적 특징



- **양방향 어텐션 (Two-way Attention):** 각 셀(cell)이 자신이 속한 **행(row)**의 다른 특징들과 어텐션을 수행하고, 동시에 자신이 속한 **열(column)**의 다른 샘플들과도 어텐션을 수행한다.

⇒ 이를 통해 샘플과 특징의 순서에 구애받지 않고 더 효율적인 학습이 가능

속도 및 메모리 최적화

- **추론 상태 캐싱 (Caching):** 매번 훈련 데이터를 다시 계산하는 비효율을 막기 위해, 훈련 데이터에 대한 계산 결과를 저장(캐싱)했다가 여러 테스트 샘플에 재사용한다.
⇒ 이를 통해 CPU에서 최대 800배의 경이로운 추론 속도 향상을 달성
- **최신 기술 집약:** Flash Attention, 활성화 체크포인트링 등 최신 최적화 기법을 총동원하여 메모리 요구사항을 4배로 줄여, 일반 상용 GPU에서도 수천만 개 셀을 가진 데이터셋을 처리할 수 있다.

Synthetic data based on causal models

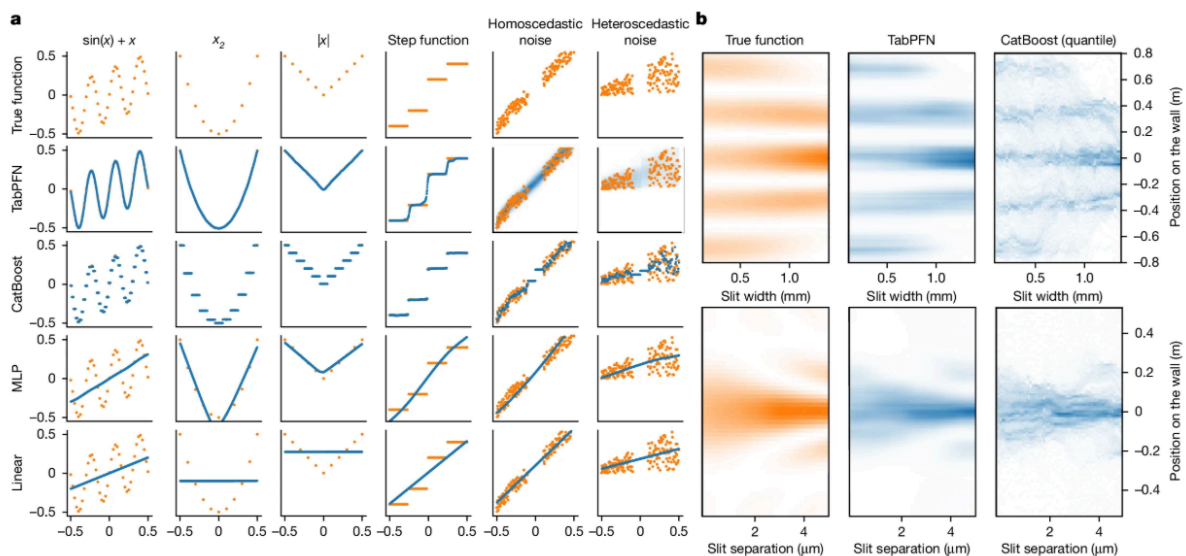
데이터 생성 파이프라인

- **인과 구조 모델 (SCM) 기반 설계:** 데이터가 생성되는 근본적인 인과 관계를 수학적 그래프로 모델링하여, 특징들 간의 단순한 상관관계를 넘어 복잡하고 비선형적인 관계를 가진 데이터셋을 무한히 생성한다.
- **현실 세계 문제 주입:** 생성 과정에서 의도적으로 결측치, 이상치(outlier), 노이즈 등을 포함시킨다. TabPFN은 이런 '더러운' 데이터로 예측하는 훈련을 반복하며, 실제 데이터의 문제를 해결하는 강건한 전략을 자율적으로 학습한다.

합성 데이터의 이점

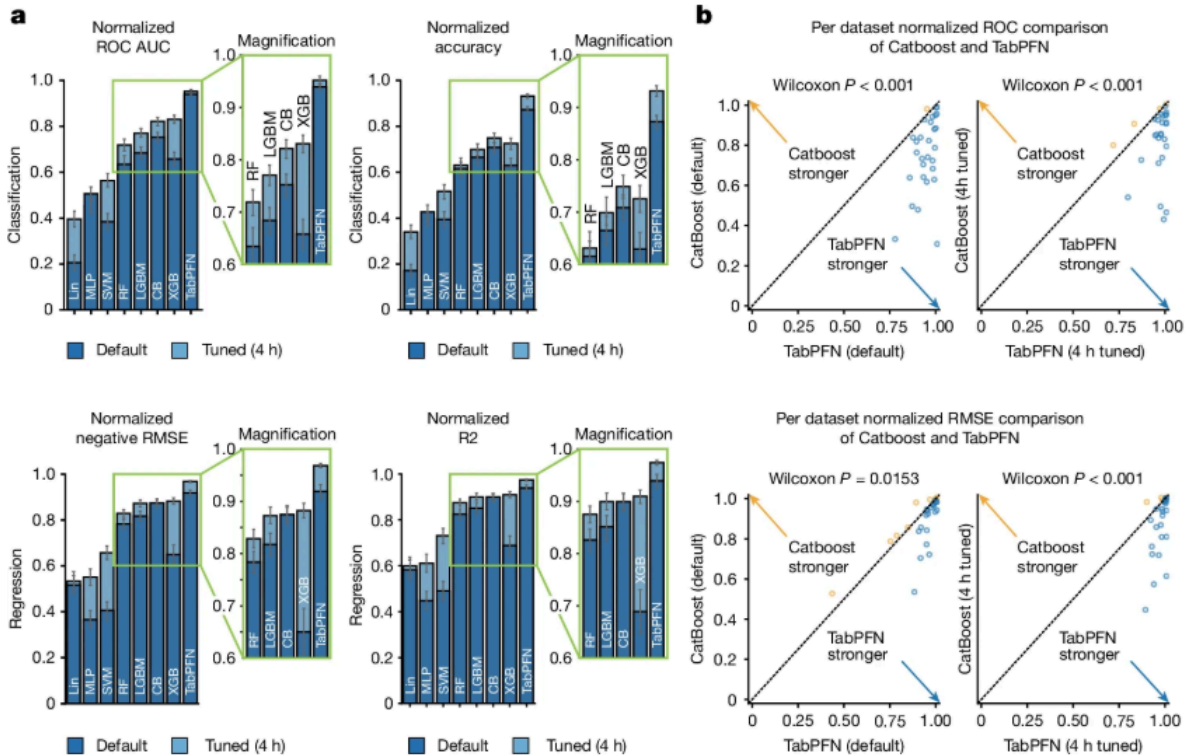
실제 데이터를 사용하는 대신 합성 데이터에 의존함으로써 개인정보 침해, 저작권, 데이터 오염과 같은 파운데이션 모델의 고질적인 문제들을 원천적으로 회피한다.

Qualitative analysis



- **함수 형태에 대한 유연성(a):** 다른 모델들이 특정 형태의 함수(예: 계단 함수)에서 취약점을 보이는 반면, TabPFN은 별도 설정 없이 **매끄러운 함수와 비-매끄러운 함수 모두**를 즉시 모델링하는 유연성을 보여준다.
- **불확실성 예측 능력(b):** 대부분의 모델이 단일 예측값을 출력하는 것과 달리, TabPFN은 추가 비용 없이 예측의 불확실성을 포함한 확률 분포를 반환한다. 이중 슬릿 실험 예제에서 볼 수 있듯, 여러 개의 봉우리를 가진 복잡한 다중 모드(multi-modal) 분포까지 정확하게 예측해낸다.

Quantitative analysis

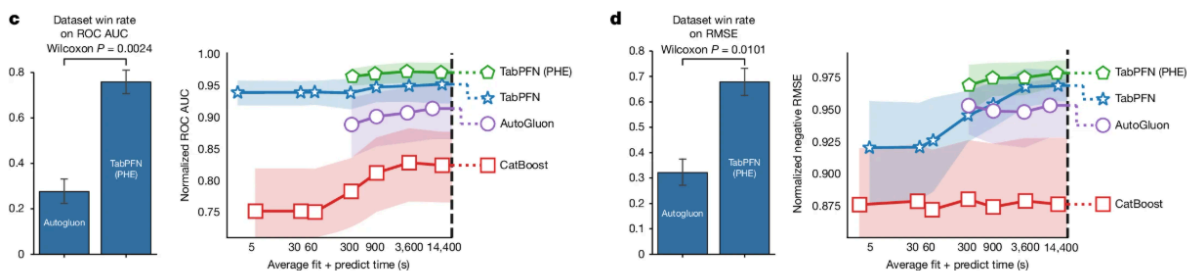


• SOTA 모델과의 비교 (a)

성능과 속도의 압살: 분류 문제에서, 단 2.8초 만에 예측을 완료한 TabPFN(기본 설정)이 4시간 동안 하이퍼파라미터 튜닝을 거친 XGBoost, CatBoost 등의 앙상블 모델보다 더 높은 성능을 기록했다. (약 5,140배의 속도 향상)

• 다양한 데이터 속성에 대한 강건성 (b)

어떤 상황에서도 강인함: 정보 없는 특징, 이상치(outlier)가 섞여 있거나 샘플 수가 절반으로 줄어드는 등, 딥러닝 모델에 일반적으로 취약하다고 알려진 상황에서도 매우 강건한 성능을 유지했다. 데이터셋의 특징(결측치 유무, 특징 수 등)에 거의 영향을 받지 않았다.

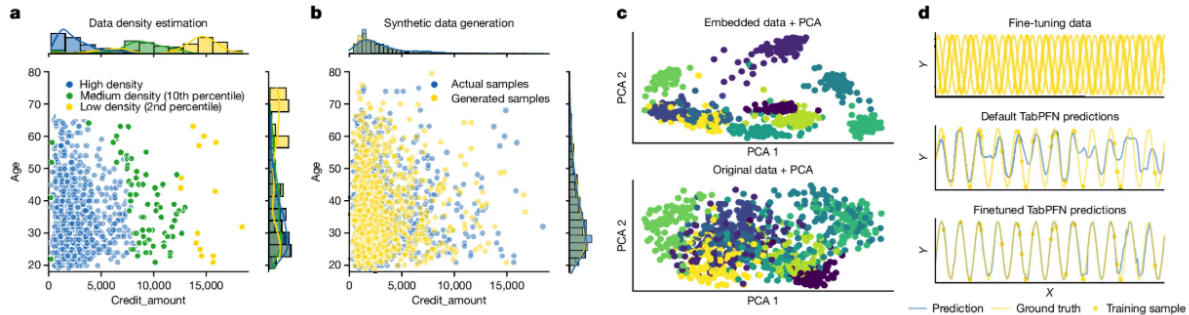


• AutoML 앙상블과의 비교

AutoGluon 능가: AutoGluon과 같은 강력한 AutoML 앙상블 기법과 비교했을 때도, TabPFN(기본)은 훨씬 짧은 시간에 더 나은 성능을 보였고, TabPFN 모델들만으로 양

상블을 구성했을 때는 AutoGluon의 성능을 크게 뛰어넘었다.

Foundation model with interpretability



• 다양한 능력

- 1) 밀도 추정: 데이터의 확률 밀도를 추정하여 사기 탐지, 이상 탐지 등에 활용할 수 있다.
- 2) 데이터 생성: 실제 데이터의 특성을 모방한 새로운 데이터를 합성하여 데이터 증강 등에 사용할 수 있다.
- 3) 임베딩 학습: 다운스트림 작업에 재사용 가능한 의미 있는 특징 표현 (representation)을 학습한다.
- 4) 미세 조정 (Fine-tuning): 특정 도메인의 데이터셋에 미세 조정을 통해 성능을 더욱 향상시킬 수 있다.

• 해석 가능성

SHAP과 같은 기법을 통해 각 특징이 예측에 미친 영향을 계산하여, 모델의 결정을 해석하고 신뢰성을 높일 수 있다.

TabNet vs TabPFN 비교

	TabNet	TabPFN
핵심 아이디어	순차적 어텐션으로 피쳐 선택	수백만 합성 데이터로 사전 학습된 파운데이션 모델
학습 방식	개별 데이터셋마다 학습	한 번 학습 → 바로 적용
속도	일반적인 DNN 학습 시간 필요	2.8초로 즉시 추론
모델 성격	특수 아키텍처 DNN	파운데이션 모델 (GPT/BERT와 같은 개념)